



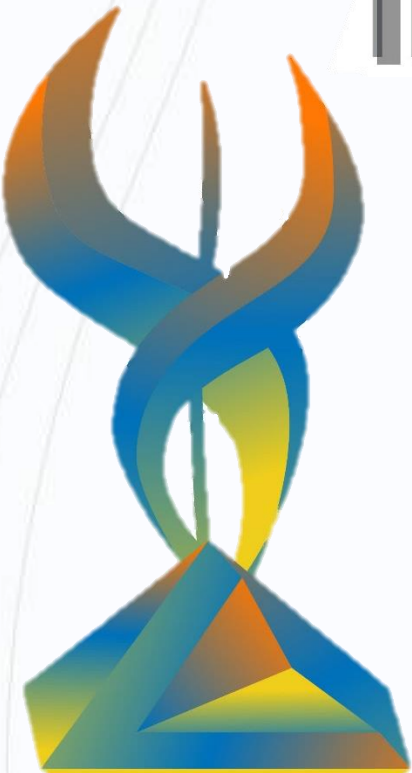
Republic of the Philippines
Bicol University
POLANGUI CAMPUS
Polangui, Albay



P3: BACK-END DEVELOPMENT & API INTEGRATION

GROUP YOYO

Llona, Yasmien Mae M.
Lusantas, Maryan D.
Ruiz, Catlyn L.
Fermin, Yasmin M.
Bron, Niña Jezreen M.
Idanan, John Mark C.



OVERVIEW OF BACKEND SETUP

1. Technology Stack

- Node.js with Express framework for the server.
- MongoDB with Mongoose ODM for database operations.
- JSON Web Tokens (JWT) for authentication.
- bcrypt for password hashing.
- Middleware for authentication and request parsing.

2. Project Structure

- server.js: Entry point of the backend server. Sets up Express app, connects to MongoDB, configures middleware (CORS, body-parser), and mounts route handlers.
- config/db.js: Handles MongoDB connection using Mongoose.
- routes/: Contains route definitions for different resources:
 - ✚ userRoutes.js: User registration and login.
 - ✚ productRoutes.js: Product listing and adding products.
 - ✚ reviewRoutes.js: Submitting and fetching product reviews.
 - ✚ cartRoutes.js: Managing shopping cart items.
 - ✚ orderRoutes.js: Placing orders and order management.
- controllers/: Contains controller logic for each resource, handling business logic and database interactions.
- models/: Mongoose schemas and models for User, Product, Review, CartItem, and Order.
- middleware/authenticateToken.js: Middleware to verify JWT tokens and protect routes.
- package.json: Lists dependencies and scripts for running the server.

3. Authentication

- Users register and login via /api/users routes.
- JWT tokens are issued on login and required for protected routes (adding products, managing cart, placing orders, submitting reviews).
- authenticateToken middleware verifies tokens on protected routes.

4. API Endpoints:

- /api/users: Register and login.

- /api/products: Get products, add product (admin only).
- /api/reviews: Submit and get reviews.
- /api/cart: Get, add, remove cart items.
- /api/orders: Place orders, get orders, cancel orders.

5. Security:

- Passwords are hashed with bcrypt.
- Role-based access control is implied (e.g., admin role for adding products).
- JWT tokens secure protected endpoints.

CODE SNIPPETS OF SERVER.JS, SAMPLE MODEL, AND ROUTES

SERVER.JS

```
backend > JS server.js > ...
1  require('dotenv').config();
2  ....
3  const express = require('express');
4  const cors = require('cors');
5  const bodyParser = require('body-parser');
6  const connectDB = require('./config/db');
7  const userRoutes = require('./routes/userRoutes');
8
9  const app = express();
10 const PORT = process.env.PORT || 3001;
11
12 connectDB();
13
14 app.use(cors());
15 app.use(bodyParser.json());
16
17 app.use('/api/users', userRoutes);
18
19 const productRoutes = require('./routes/productRoutes');
20 const reviewRoutes = require('./routes/reviewRoutes');
21 const cartRoutes = require('./routes/cartRoutes');
22 const orderRoutes = require('./routes/orderRoutes');
23
24 app.use('/api/products', productRoutes);
25 app.use('/api/reviews', reviewRoutes);
26 app.use('/api/cart', cartRoutes);
27 app.use('/api/orders', orderRoutes);
28
29 app.listen(PORT, () => {
30   console.log("Server running on http://localhost:" + PORT);
31 });
32
```

SAMPLE MODELS

backend > models > JS Product.js > ...

```
1  const mongoose = require('mongoose');
2
3  const productSchema = new mongoose.Schema({
4    name: { type: String, required: true },
5    category: String,
6    price: { type: Number, required: true },
7    image: String,
8  });
9
10 module.exports = mongoose.model('Product', productSchema);
11
```

backend > models > JS User.js > ...

```
1  const mongoose = require('mongoose');
2
3  const userSchema = new mongoose.Schema({
4    fullName: { type: String, required: true },
5    address: { type: String, required: true },
6    email: { type: String, required: true, unique: true },
7    password: { type: String, required: true },
8    role: { type: String, enum: ['user', 'admin'], default: 'user' }
9  });
10
11 module.exports = mongoose.model('User', userSchema);
12
```

backend > models > JS Order.js > ...

```
1  const mongoose = require('mongoose');
2
3  const orderItemSchema = new mongoose.Schema({
4    productId: { type: mongoose.Schema.Types.ObjectId, ref: 'Product', required: true },
5    quantity: { type: Number, required: true },
6    price: { type: Number, required: true },
7  });
8
9  const orderSchema = new mongoose.Schema({
10   userId: { type: mongoose.Schema.Types.ObjectId, ref: 'User', required: true },
11   orderNumber: { type: String, required: true, unique: true },
12   status: { type: String, required: true },
13   total: { type: Number, required: true },
14   deliveryAddress: { type: String, required: true },
15   paymentMethod: { type: String, required: true },
16   shippingMethod: { type: String, required: true },
17   orderDate: { type: Date, default: Date.now },
18   items: [orderItemSchema],
19 });
20
21 module.exports = mongoose.model('Order', orderSchema);
22
```


POST ⌵ http://localhost:3001/api/products Send

Query Headers ³ Auth Body ¹ Tests Pre Run

JSON XML Text Form Form-encode GraphQL Binary

JSON Content Format

```
1 {
2   "name": "NZXT H510 Elite",
3   "category": "PC Case",
4   "price": 89.99,
5   "image": "images/gc1.jpg"
6 }
```

Status: 201 Created Size: 27 Bytes Time: 71 ms

Response Headers ⁷ Cookies Results Docs {} ≡

```
1 {
2   "message": "Product added"
3 }
```

POST ⌵ http://localhost:3001/api/orders Send

Query Headers ³ Auth Body ¹ Tests Pre Run

JSON XML Text Form Form-encode GraphQL Binary

JSON Content Format

```
1 {
2   "deliveryAddress": "San Jose, Libon Albay",
3   "paymentMethod": "Credit Card",
4   "shippingMethod": "Standard",
5   "items": [
6     {
7       "productId": "6804c242dda2b7718ea2f25",
8       "price": 89.99,
9       "quantity": 2
10    }
11  ]
12 }
```

Status: 201 Created Size: 52 Bytes Time: 5.97 s

Response Headers ⁷ Cookies Results Docs {} ≡

```
1 {
2   "message": "Order placed",
3   "orderNumber": "NG-469740"
4 }
```

POST ⌵ http://localhost:3001/api/cart Send

Query Headers ³ Auth Body ¹ Tests Pre Run

JSON XML Text Form Form-encode GraphQL Binary

JSON Content Format

```
1 {
2   "productId": "64b8f1a2c9e77a0012345678",
3   "quantity": 2
4 }
```

Status: 200 OK Size: 26 Bytes Time: 111 ms

Response Headers ⁷ Cookies Results Docs {} ≡

```
1 {
2   "message": "Cart updated"
3 }
```

POST ⌵ http://localhost:3001/api/reviews Send

QueryHeaders³AuthBody¹TestsPre Run

JSONXMLTextFormForm-encodeGraphQLBinary

JSON ContentFormat

1 { "productId": "64b8f1a2c9e77a0012345678",
"rating": 5, "text": "Excellent product!" }

Status: 201 CreatedSize: 30 BytesTime: 83 ms

ResponseHeaders⁷CookiesResultsDocs

{ } ≡

1 {
2 "message": "Review submitted"
3 }

MEMBERS' ROLE REFLECTIONS

Member	Role
Yasmien Mae M. Llona	Project Manager
Maryan D. Lusantas	Frontend Developer
Catlyn L. Ruiz	Backend Developer
Yasmin M. Fermin	Database Manager
Niña Jezreen M. Bron	Github Manager
John Mark C. Idanan	Tester/Debugger/Documentation Officer

YASMIEN MAE M. LLONA



Project Manager

As the **Project Manager**, I made sure everyone in the team understood their roles and followed the plan for backend development. I helped organize our tasks and kept track of our progress to meet the deadline. I also supported team members when they had questions or needed help. Overall, I made sure the backend phase went smoothly and everyone worked together effectively.

As the **Frontend Developer** of group YOYO, I focused on building the user interface of our web application based on our approved design. I completed most of the user pages and I am currently working on the admin pages. During the process, I asked for help from Catlyn, our Backend Developer, especially in preparing the forms for backend integration.

MARYAN D. LUSANTAS



Frontend Developer

CATLYN L. RUIZ



Backend Developer

As a **Backend Developer**, I helped set up the Express server and connected our app to MongoDB using Mongoose. I created API routes for user data and tested them using Postman to make sure they worked properly. I also organized the code into folders like routes, models, and controllers to keep everything clean and easy to manage. My work helped lay the foundation for the rest of the project.

During Phase 3, I focused on the definition of database models using Mongoose and ensuring they properly reflected the structure of data required by the project. I established the connection between these models and MongoDB, allowing the database to effectively store and fetch the required information. I defined Mongoose models and established a stable connection to MongoDB with great care to enable smooth data operations. Through such initiatives, I ensured the backend system was clean, operational, and in working order for possible further development.

YASMIN M. FERMIN



Database Manager

NIÑA JEZREEN M. BRON



Github Manager

During Phase 3: Backend Development and API Integration, my primary contributions focused on maintaining a clean and collaborative repository as the **GitHub Manager**. I kept our repository clean and organized. By clean, I meant I avoid unnecessary commits to provide a clear history of each and everything I push.

JOHN MARK C. IDANAN



Tester/Debugger/ Documentation Officer

Phase 3 was a continuation of the system planning we completed in Phase 2. In this phase, we focused on the frontend development of our e-commerce website, NextGen Hardware. As the **Documentation Manager**, I was responsible for updating the laboratory report to reflect our latest progress. I added detailed notations for each feature, such as the Home Page, Products Page, Cart, and more, and arranged them according to the user's journey through the site.